

DL2: HOW IT WORKS? A
MULTILAYER PERCEPTRONS
(MLP'S) EXAMPLE

Recap

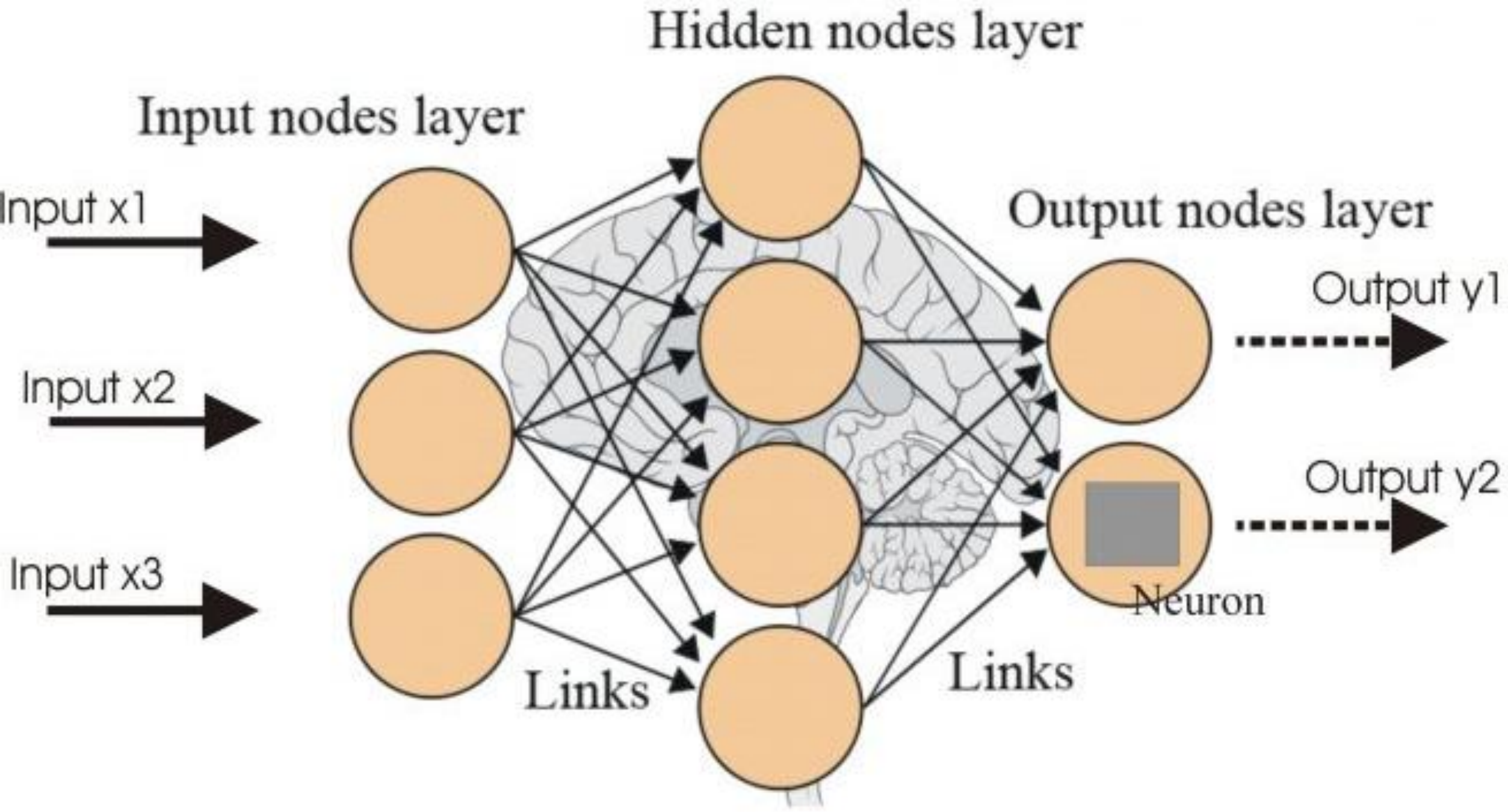
- In the previous lecture we broadly described neural networks as adaptive learning models based on the biological neuron model
- In this lecture we describe the structure of a typical deep learning model using a simple MLP example, and describe how it works
- A neural network usually consists of multiple interconnected layers, each layer consists of several neurons and feeds the next layer with some computed parameters
- Data is passed at the input layer, then through all layers up to the output layer
- In the process, the model learns how the input data is related to the expected outputs



DL Model Structure

Structure of a Deep Learning Model: At its core, a deep learning model consists of layers of interconnected nodes, or neurons, which are modeled after biological neurons. These layers are typically organized into three types:

- **Input Layer:** This is where data enters the network. Each neuron in this layer represents a feature of the input data.
- **Hidden Layers:** Between the input and output layers, there are one or more hidden layers. These layers are where the complex computations and transformations of the data take place. Deep learning involves multiple hidden layers, hence the term 'deep'.
- **Output Layer:** This layer produces the final output of the model, which could be a class label (in classification), a continuous value (in regression), or something else depending on the task.

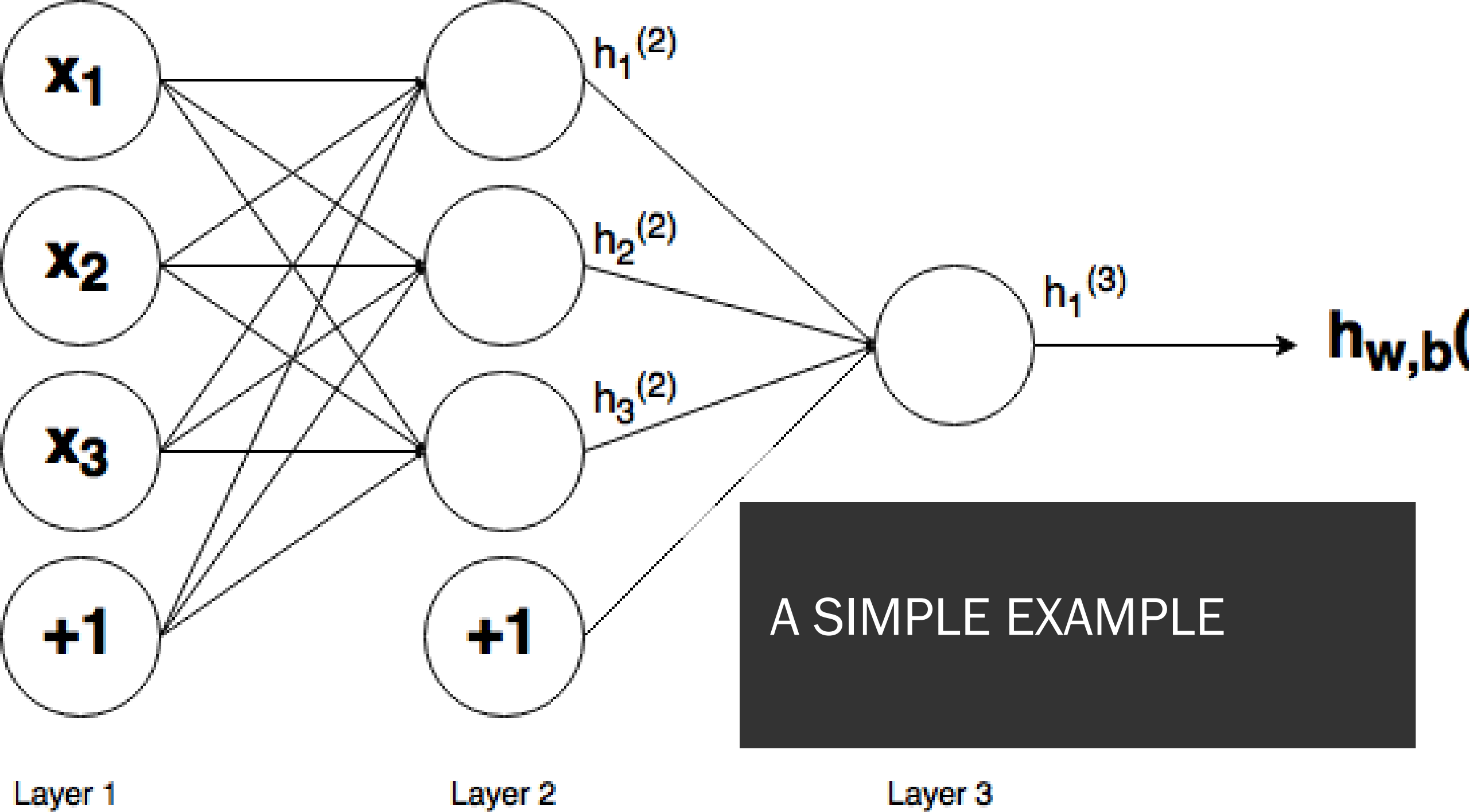


MLP Learning Process (Backpropagation)

- While the data is passed through the network, the “model's output” is compared to the “desired output”, and the difference is measured using a **loss function**.
- The model learns by adjusting the **weights** and **biases** to minimize this loss.
- This is done through a process called **backpropagation**, where the error is **propagated back** through the network, allowing the model to understand which weights and biases need to be adjusted.
- An optimization algorithm, like **stochastic gradient descent**, is used to make these adjustments in an efficient manner.

Tasks

- **Detection:** In tasks like object detection in images, the network learns to identify the presence and location of objects within an image. This involves both classifying the type of objects and localizing them within the image.
- **Classification:** In classification tasks, the network categorizes input data into predefined classes. For example, it might classify images as cats or dogs.
- **Prediction:** In predictive tasks, the network makes predictions about future data points based on learned patterns from past data. This is common in time series analysis, like stock price prediction.
- **Generation:** Generating New Novel Content



Learning

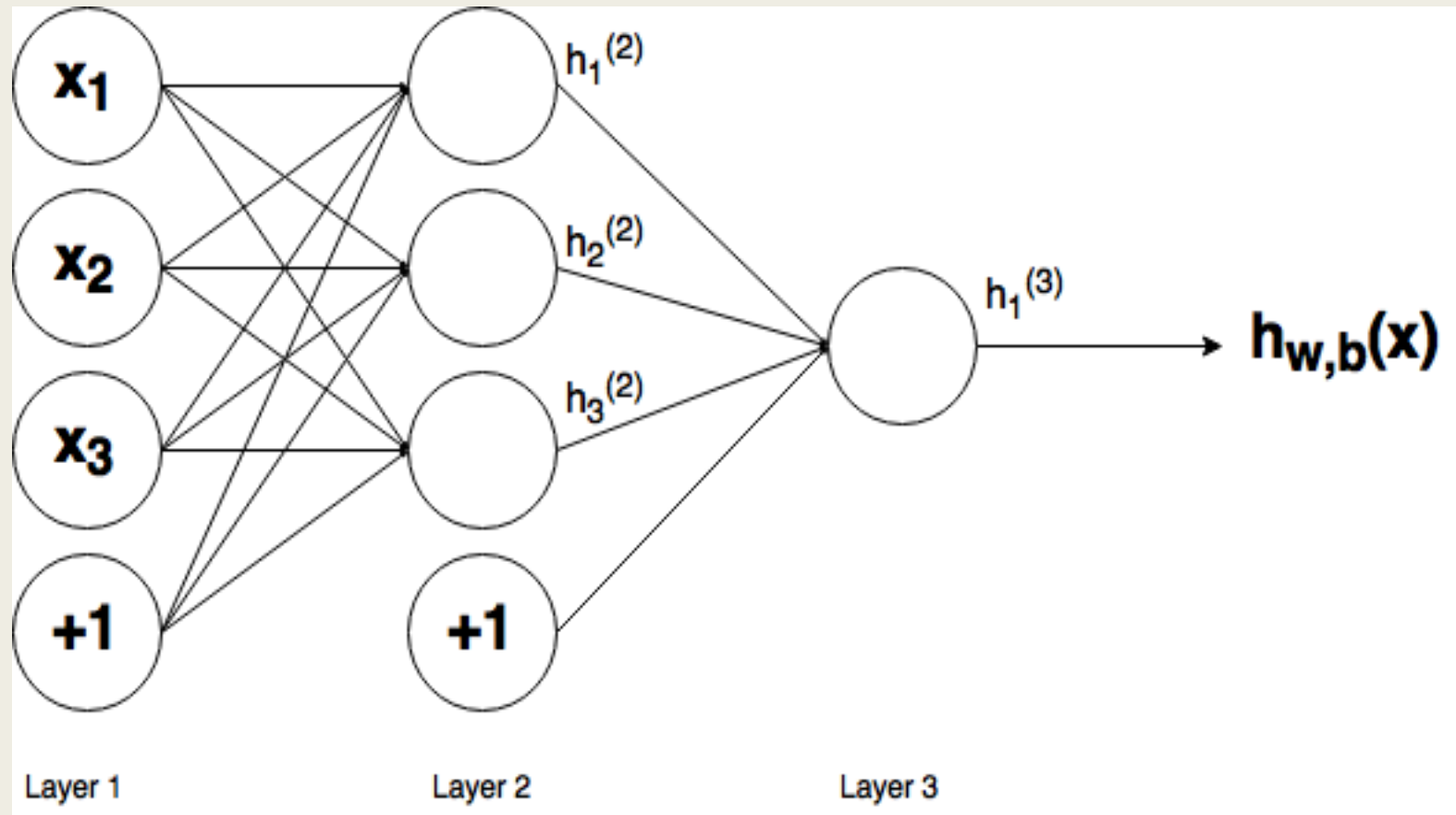
To Learn. The ANN needs:

- A Learning process
- A Learning (optimization) algorithm
- A loss (cost function) to measure the difference between actual and expected results
- An activation function to control the learning rate

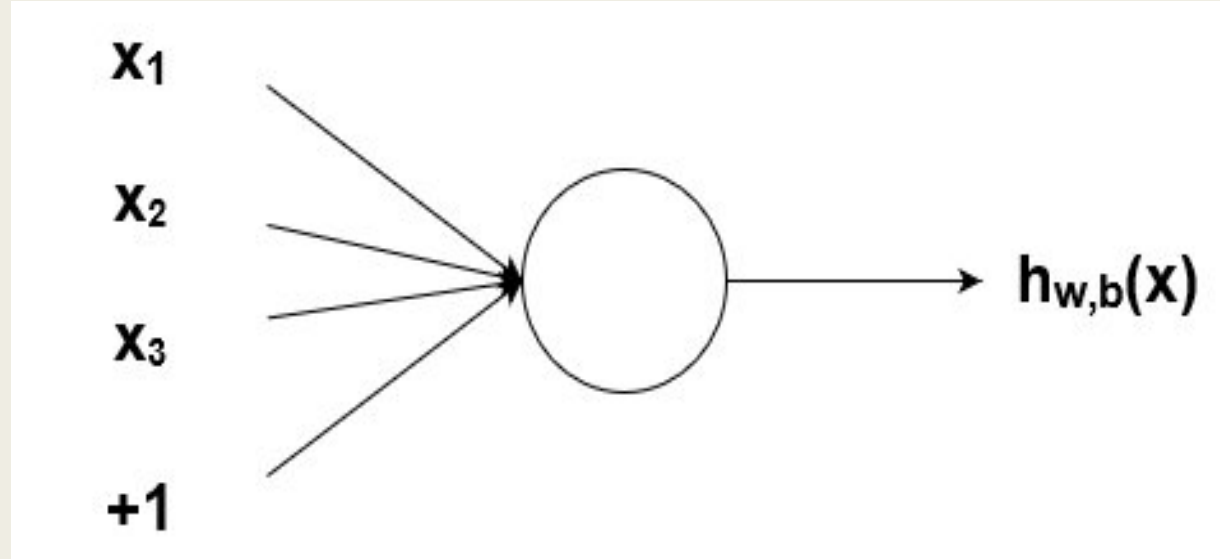
MLP's

- There are many types of neural network models – and many more will be appearing over the years
- The most popular, basic ANN model is the MLP
- Consists of one input layer, one output layer and many hidden layers (from 1 to n)

A Simple MLP with one Hidden Layer



The Perceptron

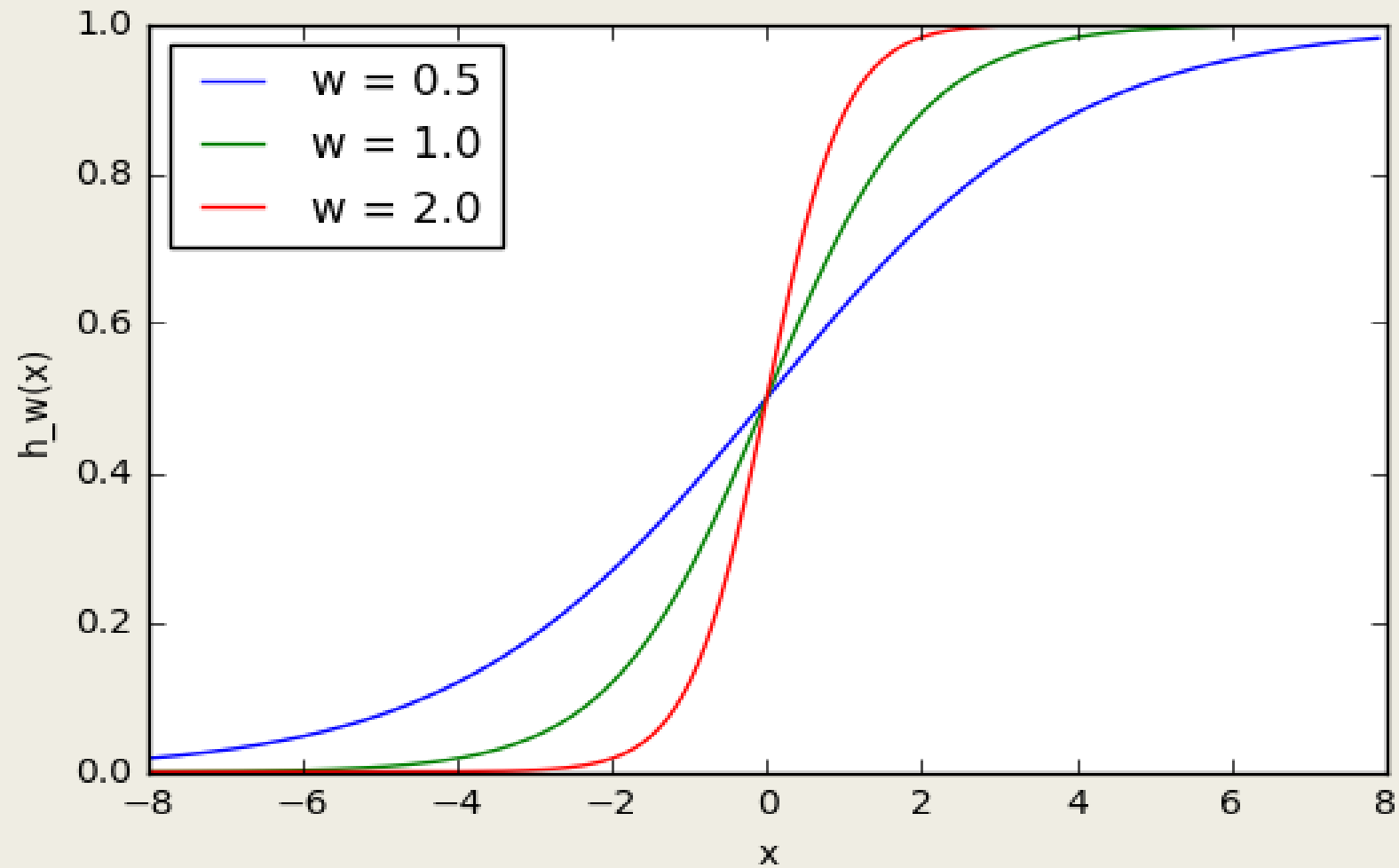


$$h = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

A Simple Activation Function for the Perceptron

```
w1 = 0.5
w2 = 1.0
w3 = 2.0
l1 = 'w = 0.5'
l2 = 'w = 1.0'
l3 = 'w = 2.0'
for w, l in [(w1, l1), (w2, l2), (w3, l3)]:
    f = 1 / (1 + np.exp(-x*w))
    plt.plot(x, f, label=l)
plt.xlabel('x')
plt.ylabel('h_w(x)')
plt.legend(loc=2)
plt.show()
```

Activation function plot



Learning

- Learning is achieved by adaptively modifying the weight based on the value of the Loss ,Function (difference between the target value and the actual computed value)
- The modification per weight is computed using gradient descent, an algorithm that will be explained later

Computing the Weights in MLP's

$$h_1^{(2)} = f(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_1^{(1)})$$

$$h_2^{(2)} = f(w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3 + b_2^{(1)})$$

$$h_3^{(2)} = f(w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3 + b_3^{(1)})$$

$$h_{W,b}(x) = h_1^{(3)} = f(w_{11}^{(2)}h_1^{(2)} + w_{12}^{(2)}h_2^{(2)} + w_{13}^{(2)}h_3^{(2)} + b_1^{(2)})$$

Layer 1 Weights

$$W^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \end{pmatrix}$$

Python NumPy Representation

```
import numpy as np  
w1 = np.array([[0.2, 0.2, 0.2], [0.4, 0.4, 0.4], [0.6, 0.6, 0.6]])
```

Layer 2 Weights

$$W^{(2)} = \begin{pmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \end{pmatrix}$$

Python NumPy Representation

```
w2 = np.zeros((1, 3))  
w2[0, :] = np.array([0.5, 0.5, 0.5])
```

The bias:

```
b1 = np.array([0.8, 0.8, 0.8])  
b2 = np.array([0.2])
```

Vectorization

$$z^{(2)} = W^{(1)}x + b^{(1)}$$

$$h^{(2)} = f(z^{(2)})$$

$$z^{(3)} = W^{(2)}h^{(2)} + b^{(2)}$$

$$h_{W,b}(x) = h^{(3)} = f(z^{(3)})$$

The upper box shows the vector representation for expressions like the one in the bottom

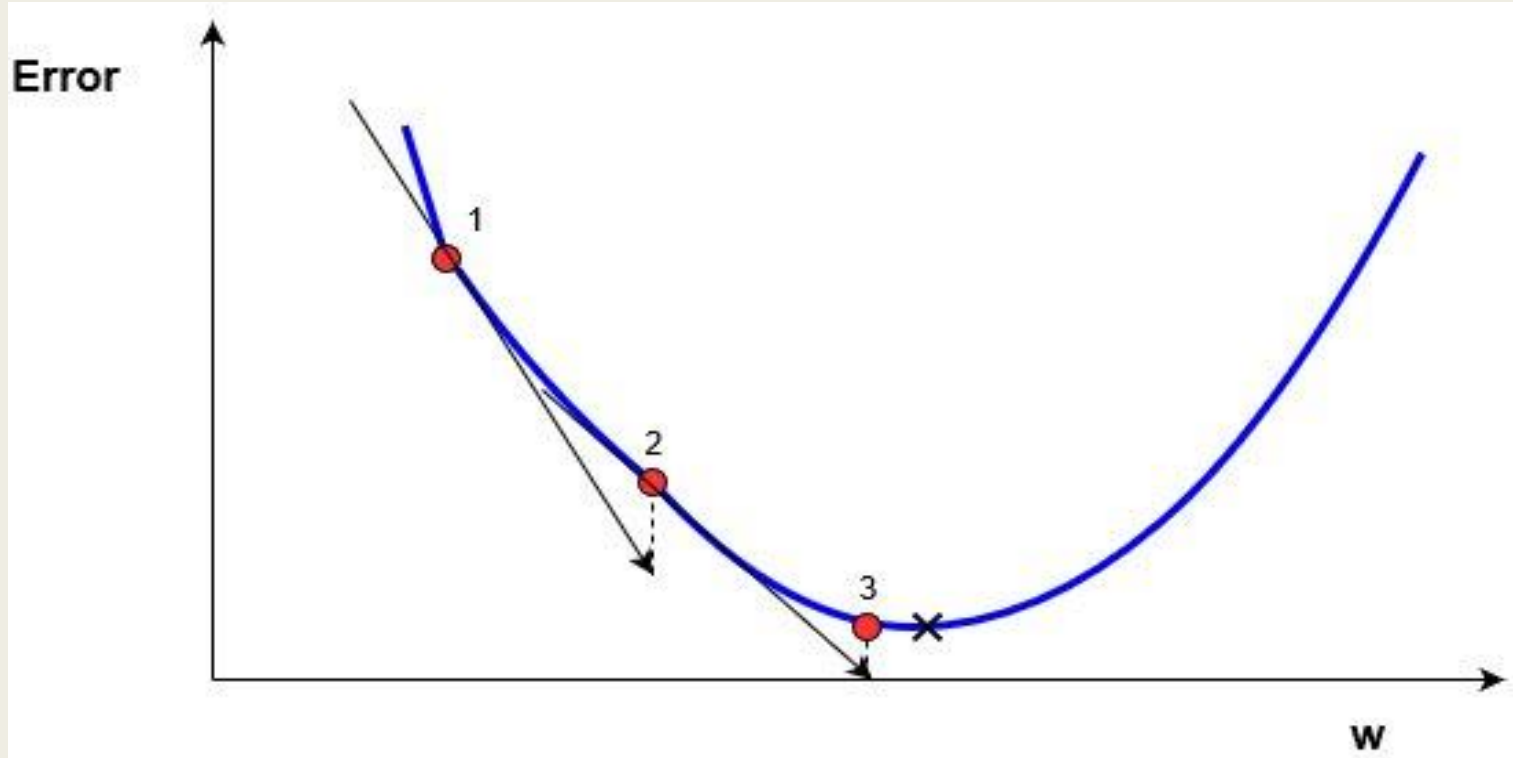
$$z_1^{(2)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_1^{(1)} = \sum_{j=1}^n w_{ij}^{(1)}x_i + b_i^{(1)}$$

Implications – Power NumPy (Tensors)

- This how matrix Multiplication in NumPy
- This is the implementation of the forward Path of our neural network

```
def matrix_feed_forward_calc(n_layers, x, w, b):  
    for l in range(n_layers-1):  
        if l == 0:  
            node_in = x  
        else:  
            node_in = h  
        z = w[l].dot(node_in) + b[l]  
        h = f(z)  
    return h
```

Gradient Descent



A Simple Example: Loss Function and Learning Algorithm

```
x_old = 0 # The value does not matter as long as abs(x_new -  
x_old) > precision  
x_new = 6 # The algorithm starts at x=6  
gamma = 0.01 # step size  
precision = 0.00001
```

```
def df(x):  
    y = 4 * x**3 - 9 * x**2  
    return y
```

```
while abs(x_new - x_old) > precision:  
    x_old = x_new  
    x_new += -gamma * df(x_old)
```

```
print("The local minimum occurs at %f" % x_new)
```